

От монолита к автономным агентам: архитектура и граф состояний ИИ-агента в 1С

Первая статья цикла из 7 практических руководств по проектированию, разработке и промышленному внедрению автономных мультиагентных систем в корпоративном контуре «1С:Предприятие 8» на примере открытого проекта NOPiK.

Целевая аудитория: Архитекторы решений 1С, ведущие разработчики, руководители ИТ-отделов, специалисты по интеграции ИИ.

Применимость: 1С:Предприятие 8.3 (не ниже 8.3.20), любая типовая конфигурация (ERP, УТ, КА, УНФ, БП, ЗУП). Точка входа подключается через сменный адаптер: «Команда формы» работает уже сегодня, адаптер на основе объекта «Бот» Системы взаимодействия - начиная с платформы 8.5.5 (пока не выпущена, см. раздел «Точка входа»).

Серия: NOPiK - автономный ИИ-агент для 1С:Предприятие 8 (статья 1 из 7).

Оглавление:

1. Кризис наивного вайб-кодинга в Enterprise
2. Архитектурный манифест проекта NOPiK
3. Точка входа: адаптерный слой вместо жесткой привязки к объекту «Бот»
4. Теория управления агентом: FSM и HSM
5. Репозиторий, Git и ibcmd
6. Модуль бота в 1С
7. Учебная ветка: свой оркестратор на Python / LangGraph
8. Дорожная карта цикла

Введение: кризис наивного вайб-кодинга в Enterprise

Первая волна энтузиазма вокруг больших языковых моделей (LLM) в сообществе 1С породила десятки однотипных решений: внешняя обработка, на форму которой выведено поле ввода промпта, многострочное текстовое поле ответа и кнопка, отправляющая синхронный HTTP-запрос к публичному API нейросети.

В реальном корпоративном секторе такой подход терпит немедленный крах по четырем причинам.

1. **Иллюзия агентности (Stateless Predictor vs Stateful Agent).** Сама по себе языковая модель - это статический вероятностный предсказатель следующего токена. У нее нет ни долгосрочной памяти, ни воли, ни внутреннего детерминированного состояния. Попытка возложить на один гигантский промпт («Ты ведущий эксперт 1С, проверь остатки, сформируй проводки и создай документ») приводит к катастрофическому росту

контекста, галлюцинациям в именах реквизитов и нарушению ссылочной целостности базы.

2. **Закон Люссера: падение надежности цепочки шагов.** Согласно закону Люссера, надежность последовательной системы равна произведению надежностей каждого ее элемента: $R_{sys} = R_1 \times R_2 \times \dots \times R_n$. Если на каждом шаге вероятность корректного ответа модели составляет даже 90% (0.9), то цепочка всего из 5 невалидируемых шагов завершится успехом с вероятностью менее 60% ($0.9^5 \approx 0.59$). В контуре бухгалтерского или складского учета такая погрешность недопустима: за долей несошедшихся цепочек стоят не абстрактные проценты, а неверные проводки и расхождения в остатках.
3. **Хрупкость кастомных интерфейсов (UI Fragmentation).** Попытки встроить в форму 1С веб-документ (HTML/JS/React WebView) для имитации чата приводят к постоянным сбоям: несовместимость движков рендеринга на разных ОС (Windows, Linux-дистрибутивы Astra/RED OS), проблемы с пробросом событий через `document.location` и сломанная верстка при обновлении платформы.
4. **Нарушение транзакционной изоляции.** Синхронное ожидание ответа модели внутри сеанса пользователя 1С блокирует интерфейс, вызывает клиентские таймауты и создает риск зависания транзакций при сетевых лагах.

Чтобы преодолеть эти ограничения, необходим переход от монолитных скриптов к системной инженерии автономных агентов, где модель помещена внутрь строгого математического автомата состояний, отделена от прикладных данных протокольными контрактами и встроена в нативные механизмы платформы.

Архитектурный манифест: проект NOPik

В качестве сквозного демонстрационного проекта цикла мы берем архитектуру **NOPik** - корпоративного ИИ-агента. Исходный код расширения и примеры оркестратора публикуются в открытом репозитории проекта.

Фундаментальный принцип NOPik - **«Zero-Footprint Core»** (минимальный след в целевой конфигурации): внутри контура 1С:Предприятие живет только точка входа - сменный адаптер и пара обслуживающих модулей - а вся оркестрация, память и рассуждение вынесены за периметр учетной системы. Сам адаптер не фиксирован жестко: единый программный контракт приема сообщений реализован в общем модуле, а способ его физического вызова - через объект «Бот» или через команду формы - подключается отдельно (подробности - в разделе «Точка входа» ниже).

Контур 1С:Предприятие (Zero-Footprint)

Пользователь (тонкий / веб / мобильный клиент) → адаптер точки входа (сегодня - команда формы; с 8.5.5 - объект «Бот» Системы взаимодействия) → единый контракт приема сообщений (~30 строк BSL)



HTTP POST: JSON Payload + TraceID

Реальный внешний сервис (nopikreport.com/api/ask)

Синхронный RAG-ответ на вопрос (Vertex AI Discovery Engine) - живой, работающий сервис, а не часть кода этого цикла



запросы к данным через Tools

Протокольный контур MCP (Model Context Protocol)

MCP-сервер 1С: метаданные, безопасное чтение через СКД

Схема. Три контура архитектуры NOPik и точки пересечения по сети

Такое разделение убирает из контура 1С любую бизнес-логику, специфичную для ИИ, и позволяет обновлять «мозг» агента, не трогая продуктивную базу.

📁 Точка входа: адаптерный слой вместо жесткой привязки к объекту «Бот»

В платформе «1С:Предприятие 8.3» существует штатный объект дерева метаданных **Общие → Боты**. У него есть реальные преимущества как у точки входа: нативность (пользователь общается с агентом в стандартном окне обсуждений, рядом с коллегами и привязанными к документам чатами - одинаково в тонком клиенте на Windows и Linux, в браузере и в мобильном клиенте) и готовый контекст обсуждения (бота можно тегать через @NOPik в чате конкретного заказа, где агент сразу получает идентификаторОбсуждения и ссылку на объект).

Но у этого преимущества есть срок начала действия, который честно нужно назвать: **для расширений конфигурации (CFE) объект «Бот» становится доступен только начиная с платформы 8.5.5** - на момент публикации эта версия еще не вышла (по объявленному 1С плану выпуск ожидается во второй половине 2026 года, а массовое распространение на типовых конфигурациях - позже). До этого момента объект «Бот» существует только в

основной конфигурации (CF), а не в расширении, и жесткая привязка к нему в расширении делает цикл нерабочим прямо сейчас. Проверено эмпирически на платформе 8.3.27.2130: пункт «Добавить» для объекта «Боты» в дереве расширения недоступен независимо от режима совместимости.

Решение - не ждать 8.5.5 и не снимать конфигурацию с поддержки ради обхода, а **развязать точку входа от бизнес-логики через адаптерный слой**. Весь прием сообщения сведен к одному программному контракту в общем модуле `НОПик_Сеть`:

```
НОПик_Сеть.ПоставитьЗадачуНаОтправку(АвторID, ТекстСообщения, ИдентификаторОбсуждения = "")
```

К этому контракту сегодня подключены два адаптера, каждый из которых лишь извлекает (автора, текст, идентификатор обсуждения) из своего родного события и передает их дальше без дублирования логики:

- **«Команда формы» (доступен уже сейчас)**. Общая команда `НОПик_СпроситьАгента` запрашивает у пользователя текст сообщения стандартным диалогом ввода строки и передает его в тот же контракт. Не требует объекта «Бот», работает на любой поддерживаемой версии платформы, устанавливается и проверяется в тестовой базе за минуты.
- **«Бот» Системы взаимодействия (с 8.5.5)**. Код обработчика сообщений уже написан и лежит в расширении - он активируется как только целевая платформа и конфигурация подтвердят поддержку 8.5.5, без изменения контракта и без переписывания оркестратора.

Такое разделение - пример разрешения противоречия «точка входа должна быть объектом «Бот» и одновременно не может им быть» через отделение способа доставки сообщения от бизнес-логики его обработки, а не через компромисс или ожидание. Один и тот же прием применяется в цикле еще как минимум дважды: в изоляции MCP-контура от специфики 1С и в Circuit Breaker, отделяющем доступность сети от бизнес-логики (см. статьи 3 и 5).

Теория управления агентом: граф состояний FSM и HSM

Отличие Predictor от Agent

Агентная система возникает только тогда, когда рассуждение модели связывается с циклом действий и обратной связи из среды: $c(t+1) = (c_t, a_t, o_t)$, где a_t - действие в среде (вызов инструмента/API) или рассуждение (мысль/план), а o_t - наблюдаемый отклик среды (ошибка компиляции, результат SQL/BSL-запроса, ответ пользователя).

Чтобы агент не уходил в бесконечные циклы и сохранял управляемость, его поведение моделируется в виде **конечного автомата состояний** (Finite State Machine, FSM), описываемого кортежем $M = (S, \Sigma, \delta, s_0, F)$, где:

- S - конечное множество состояний агента (например: RECEIVE, PLAN, CALL_TOOL, VERIFY, RESPOND);
- Σ - входной алфавит событий (получено сообщение, инструмент вернул данные, ошибка валидации, отказ пользователя);
- $\delta: S \times \Sigma \rightarrow S$ - функция переходов между состояниями с предикатами защиты (Guards);
- $s_0 \in S$ - начальное состояние (IDLE / RECEIVE);
- $F \subseteq S$ - терминальные состояния (RESPOND, ABORT).

Переход к иерархическим автоматам (HSM)

Для сложных сценариев плоский FSM приводит к взрывному росту числа связей («спагетти-маршрутизация»). В архитектуре NOPIK мы используем **иерархический автомат состояний** (Hierarchical State Machine, HSM). В HSM состояния могут быть сгруппированы в суперсостояния (Superstates):

- **Суперсостояние** WORKING объединяет подсостояния ANALYZE_METADATA, COMPOSE_QUERY, EXECUTE_READ.
- **Общие политики (Global Guards)**. Лимиты токенов, тайм-ауты транзакций и контуры безопасности (Circuit Breakers) вешаются на суперсостояние один раз и наследуются всеми дочерними узлами.
- **Чекпоинтинг (Deep History)**. Маркер состояния сохраняет снимок контекста на диск/в БД. Если на шаге 4 из 6 произошел сбой сети или падение сервиса, выполнение возобновляется ровно с контрольной точки шага 4, не требуя повторных затрат на предшествующие вызовы LLM.



Инфраструктура разработки: от монолита к Git и ibcmd

Разработка ИИ-агентов требует высокой скорости итерирования. Классическая схема «Конфигуратор → Сохранить → Обновить конфигурацию БД» здесь неприменима: она медленная и не автоматизируется скриптами агентов.

Конфигурация расширения версионизируется в формате открытых иерархических файлов (XML/JSON/BSL) — том же, что дают штатная выгрузка конфигурации в файлы средствами платформы и 1C:EDT:

Структура репозитория NOPik/

☐ NOPik/	
☐ .claude/	# Навыки и системные промпты для агентных IDE
☐ .v8-project.json	# Манифест проекта 1C
☐ Configuration.xml	# Корневой манифест метаданных конфигурации/расширения
☐ ConfigDumpInfo.xml	# Служебная информация о версиях объектов
☐ CommonModules/	
☐ НОПик_КлиентСервер/	# Общий модуль адаптера «Команда формы»
☐ НОПик_Сеть/	# Общий модуль HTTP-клиента к nopikreport.com
☐ НОПик_СКД/	# Общий модуль программного отчёта СКД
☐ CommonCommands/	
☐ НОПик_СпроситьАгента/	# Команда вызова адаптера «Форма»
☐ Constants/	
☐ НОПик_TokenAuth/	# Токен доступа к внешнему сервису
☐ HTTPServices/	
☐ НОПик_СервисСКД/	# Боевой backend для tcp-server (не адаптер входа)
☐ Bots/	
☐ НОПик/	# Объект метаданных «Бот» (доступен с 8.5.5)
☐ Bot.xml	# Описание объекта метаданных
☐ Ext/	
☐ Module.bs1	# Модуль бота (перехват сообщений)

Вместо запуска тяжелого графического Конфигуратора для применения изменений используется утилита командной строки **ibcmd**, доступная в дистрибутивах платформы 8.3. Для частичной накатки изменений из Git-репозитория в тестовую базу выполняется скрипт:

Bash — deploy_partial.sh

UTF-8

CLI

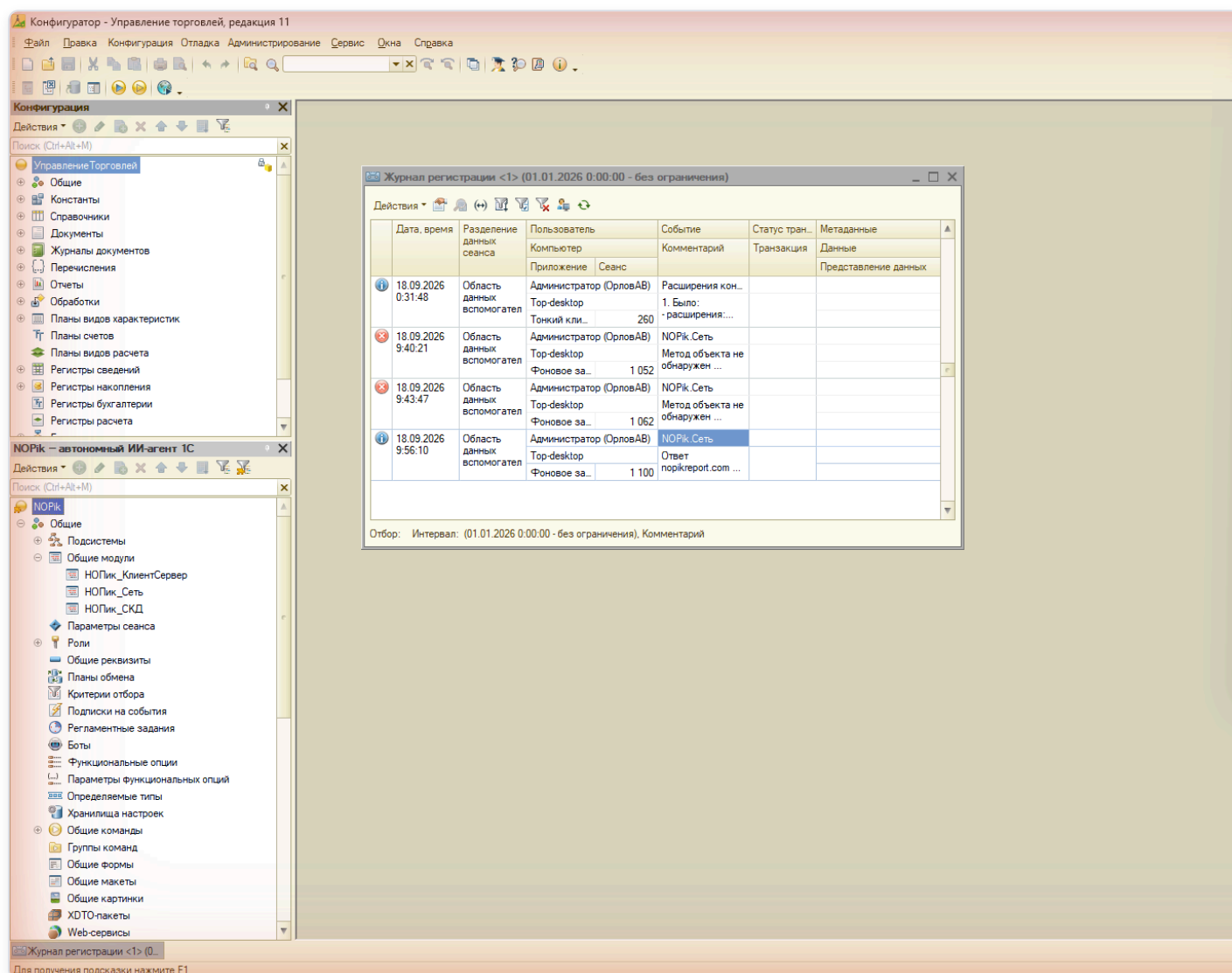
```
1  #!/usr/bin/env bash
2  set -e
3
4  IB_PATH="/var/1c/bases/test_db"
5  SRC_PATH="./src"
6
7  # Частичная загрузка измененных исходников в информационную базу
8  ibcmd infobase config import \
9      --data-dir="${IB_PATH}" \
10     --src-dir="${SRC_PATH}" \
11     --partial
12
13  echo "Изменения успешно применены через ibcmd без блокировки Конфигуратора."
```

При работе в ОС Linux (или WSL2 под Windows) критически важно задать локальную настройку Git, исключающую экранирование кириллических путей к объектам метаданных 1C: `git config core.quotePath false`.

Практическая реализация: модуль бота в 1С

Создадим расширение со свойствами:

- **Имя:** NOPic
- **Префикс:** НОПик_
- **Назначение:** Дополнение
- **Режим совместимости:** Соответствует основной конфигурации (не ниже 8.3.20).



Дерево расширения NOPic в Конфигураторе 1С: реальные общие модули (НОПик_КлиентСервер, НОПик_Сеть, НОПик_СКД) под настоящим кириллическим префиксом расширения «НОПик_», пустая ветка «Боты» - платформенное ограничение до 8.5.5, общая команда НОПик_СпроситьАгента, HTTP-сервис НОПик_СервисСКД.

Ниже - модуль адаптера «Бот»: общие → Боты → нопик в дереве метаданных расширения.

Напоминание из раздела «Точка входа» выше: этот объект физически создается в расширении начиная с платформы 8.5.5. Код приведен как целевая реализация адаптера и уже лежит в репозитории проекта; для проверки цикла на текущей платформе используйте адаптер «Команда формы» (код - сразу после этого блока).

Обратите внимание, насколько тонкий этот модуль: он не собирает контекст запроса сам, а лишь извлекает из объекта «Сообщение обсуждения» три значения (автора, текст, идентификатор обсуждения) и передает их в единый контракт общего модуля НОПик_Сеть - тот же самый, которым пользуется адаптер «Команда формы».

BSL - Bots/НОПик/Ext/Module.bsl

UTF-8

[Открыть файл](#)

```
1 // Обслуживание Системы взаимодействия для агента NOPik
2 #Область ОбработчикиСобытий
3
4 // Основная точка входа для обработки сообщений, адресованных боту
5 Процедура ОбработкаСообщения(Сообщение, Отказ) Экспорт
6
7 // Guard-клауза вынесена в НОПик_КлиентСервер отдельной чистой функцией,
8 // чтобы ее можно было покрыть модульным тестом без объекта "Сообщение
9 // обсуждения" и без полноценного сеанса 1С (см. Tests/YAxUnit)
10 Если НОПик_КлиентСервер.ЭтоСобственноеСообщениеБота(Сообщение.Автор, Пользователь)
11     Возврат;
12 КонецЕсли;
13
14 // Адаптер «Бот» лишь извлекает (автор, текст, обсуждение) из своего
15 // родного события и передает их в единый контракт НОПик_Сеть - тот же,
16 // которым пользуется адаптер «Команда формы» (см. НОПик_КлиентСервер)
17 НОПик_Сеть.ПоставитьЗадачуНаОтправку(
18     Строка(Сообщение.Автор),
19     Сообщение.Текст,
20     Строка(Сообщение.ИдентификаторОбсуждения),
21     Сообщение.Обсуждение
22 );
23
24 КонецПроцедуры
25
26 #КонецОбласти
```

А вот адаптер «Команда формы» - тот, который можно проверить прямо сейчас, без ожидания 8.5.5. Общая команда нопик_спроситьАгента запрашивает текст сообщения стандартным диалогом ввода строки и передает его через клиент-серверный модуль НОПик_КлиентСервер в тот же контракт НОПик_Сеть.ПоставитьЗадачуНаОтправку:

BSL - CommonModules/НОПик_КлиентСервер/Ext/Module.bsl (фрагмент)

UTF-8

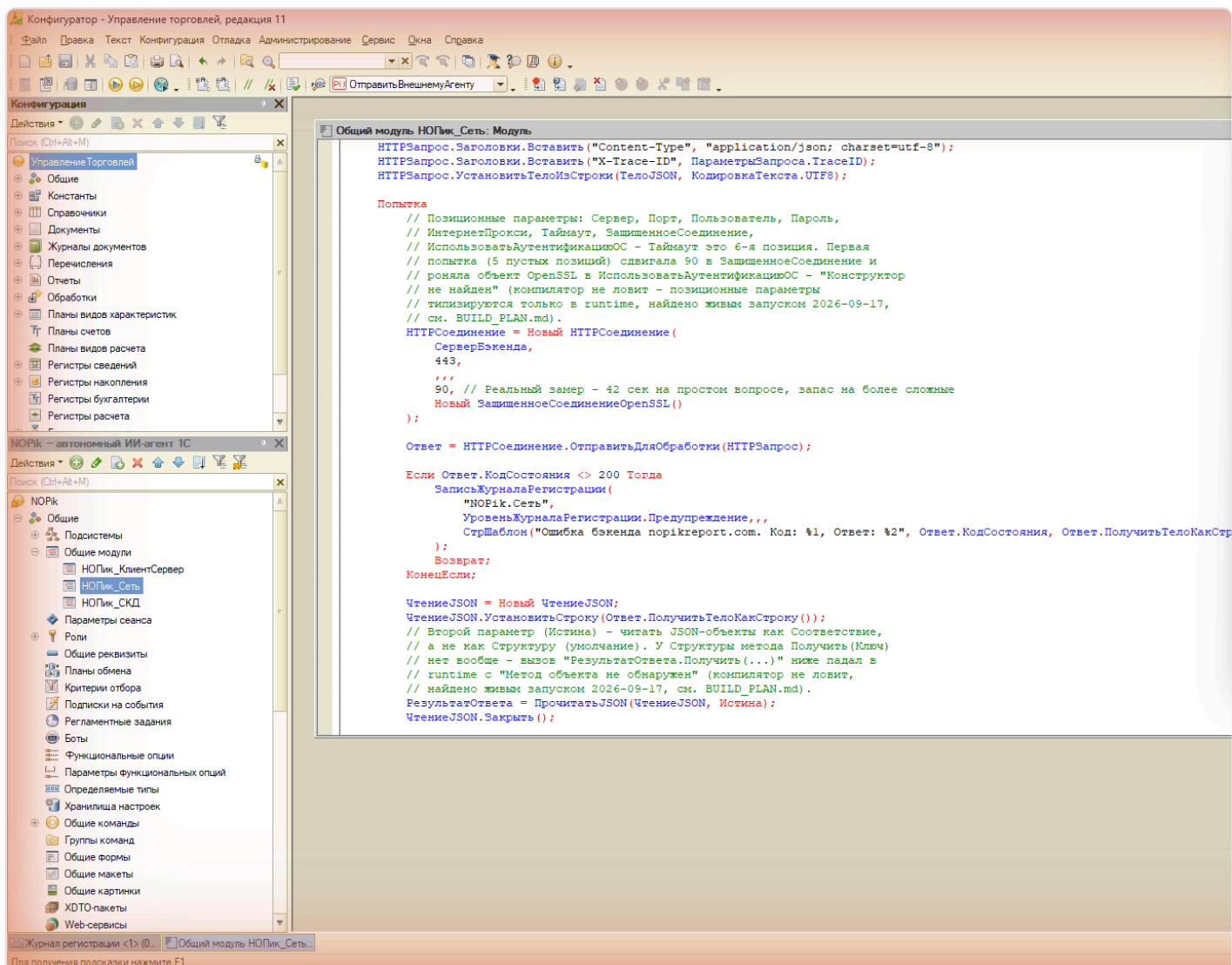
[Открыть файл](#)

```
1 // Адаптер «Команда формы»: работает на любой платформе уже сегодня, без
2 // объекта метаданных «Бот» (тот появится в расширениях только с 8.5.5)
3 &НаКлиенте
4 Процедура ЗапроситьИОтправитьСообщение() Экспорт
5     Оповещение = Новый ОписаниеОповещения("ПослеВводаСообщенияНОПик", НОПик_КлиентСервер.ПоставитьЗадачуНаОтправку);
```

```

6      ПоказатьВводСтроки(Оповещение, "", "NOPik: сообщение агенту", 500);
7      КонецПроцедуры
8
9      &НаКлиенте
10     Процедура ПослеВводаСообщенияНОПик(РезультатВвода, ДополнительныеПараметры) Экспорт
11         Если РезультатВвода = Неопределено Или ПустаяСтрока(РезультатВвода) Тогда
12             Возврат;
13         КонецЕсли;
14         НОПик_Сеть.ОтправитьСообщениеФормыНаСервере(РезультатВвода);
15     КонецПроцедуры
16
17     &НаСервере
18     Процедура ОтправитьСообщениеФормыНаСервере(ТекстСообщения) Экспорт
19         НОПик_Сеть.ПоставитьЗадачуНаОтправку(
20             Строка(ПользователиИнформационнойБазы.ТекущийПользователь()).УникальныйИдентификатор,
21             ТекстСообщения,
22             "form-adapter"
23         );
24     КонецПроцедуры

```



Процедура ОтправитьВнешнемуАгенту в Конфигураторе - финальный код после живого тестирования: второй параметр истина у ПрочитатьJSON (баг №6 из раздела «Живое тестирование» ниже).

Оба фрагмента выше показаны уже в финальном, исправленном виде. Первая версия компилировалась без единой ошибки («Проверка модулей» в Конфигураторе - синтаксический контроль, не выполнение) и выглядела корректной на бумаге. Реальный прогон в 1С:Предприятии вскрыл 6 runtime-ошибок, которые статический контроль платформы не поймал ни разу - полный разбор, с точными формулировками ошибок и скриншотами живого выполнения, в разделе «Живое тестирование» ниже. Модуль на стороне сервера (CommonModules/НОПик_Сеть/Module.bs1) обращается к настоящему, действующему сервису nopikreport.com/api/ask - без ключа авторизации, с ответом за десятки секунд (замерено живым вызовом - 42 секунды на простом вопросе), а не быстрым подтверждением приема: таймаут HTTP-соединения выставлен в 90 секунд именно поэтому. Полученный ответ фоновое задание публикует обратно в то же обсуждение Системы взаимодействия, из которого пришел вопрос.

BSL — CommonModules/НОПик_Сеть/Module.bs1

UTF-8

[Открыть файл](#)

```
1  #Область ПрограммныйИнтерфейс
2
3  // Единая точка входа контракта адаптеров: любой способ доставки сообщения в
4  // контур 1С (объект «Бот» Системы взаимодействия, общая команда формы или,
5  // в будущем, любой другой адаптер) сводится к этим параметрам и здесь
6  // становится одинаковой асинхронной задачей отправки во внешний сервис.
7  //
8  // Параметры:
9  //  АвторID          - Строка - идентификатор автора сообщения в
10 //                    терминах адаптера-вызывающей стороны.
11 //  ТекстСообщения   - Строка - текст сообщения пользователя.
12 //  ИдентификаторОбсуждения - Строка - идентификатор диалога/треда, если
13 //                    у адаптера есть такое понятие (по умолчанию "").
14 //  ОбъектОбсуждения - СправочникСсылка.Обсуждения / Неопределено -
15 //                    реальная ссылка на обсуждение, если адаптер её
16 //                    предоставляет - используется для доставки
17 //                    ответа обратно в чат. У адаптера «Команда формы»
18 //                    такого объекта нет: доставка ответа для него не
19 //                    входит в объем этой задачи, ответ только
20 //                    протоколируется в журнале регистрации.
21 Процедура ПоставитьЗадачуНаОтправку(АвторID, ТекстСообщения, ИдентификаторОбсуждения)
22
23     КонтекстЗапроса = Новый Структура;
24     КонтекстЗапроса.Вставить("MessageID",      Строка(Новый УникальныйИдентификатор));
25     КонтекстЗапроса.Вставить("ConversationID",  ИдентификаторОбсуждения);
26     КонтекстЗапроса.Вставить("AuthorID",       АвторID);
27     КонтекстЗапроса.Вставить("Text",           ТекстСообщения);
28     КонтекстЗапроса.Вставить("Timestamp",      Формат(ТекущаяУниверсальнаяДатаВМиллисекундах));
29     КонтекстЗапроса.Вставить("TraceID",        НОПик_КлиентСервер.НовыйTraceID());
30     КонтекстЗапроса.Вставить("ОбъектОбсуждения", ОбъектОбсуждения);
31
32     // Передача обработки в асинхронный фоновый контур: реальный ответ
```

```

33 // nopikreport.com приходит синхронно, но за десятки секунд (замерено
34 // живым вызовом - 42 сек на простом вопросе) - недопустимо ждать этого
35 // в сеансе пользователя.
36 ФоновыеЗадания.Выполнить(
37     "НОПик_Сеть.ОтправитьВнешнемуАгенту",
38     ПараметрыЗадания = Новый Массив; ПараметрыЗадания.Добавить(КонтекстЗапроса);
39     ПараметрыЗадания,
40     КонтекстЗапроса.TraceID,
41     "NOPik: Запрос к nopikreport.com"
42 );
43
44 КонецПроцедуры
45
46 // Процедура выполняется в фоновом задании.
47 //
48 // Реальный контракт nopikreport.com (проверено живым HTTP-вызовом
49 // 2026-09-17): POST на голый домен (без "api." поддомена - тот не отвечает
50 // по TLS), без ключа авторизации, синхронный ответ с готовым текстом за
51 // 40+ секунд, а не быстрый 202 Accepted, как предполагалось раньше.
52 Процедура ОтправитьВнешнемуАгенту(ПараметрыЗапроса) Экспорт
53
54     СерверБэкенда = "nopikreport.com";
55     Ресурс          = "/api/ask";
56
57     ТелоЗапроса = Новый Структура;
58     ТелоЗапроса.Вставить("question", ПараметрыЗапроса.Text);
59     // "utmp" - реальное значение, разбирающееся backend'ом в движок
60     // "1c-code-intelligence-engine" (код+руководства 1С). Любое нераспознанное
61     // значение backend молча трактует как продукт по умолчанию "buh" -
62     // бухгалтерский движок. Проверено сравнением источников ответа на один
63     // и тот же вопрос при разных product (2026-09-17).
64     ТелоЗапроса.Вставить("product", "utmp");
65
66     ЗаписьJSON = Новый ЗаписьJSON;
67     ЗаписьJSON.УстановитьСтроку();
68     ЗаписатьJSON(ЗаписьJSON, ТелоЗапроса);
69     ТелоJSON = ЗаписьJSON.Закрыть();
70
71     HTTPЗапрос = Новый HTTPЗапрос(Ресурс);
72     HTTPЗапрос.Заголовки.Вставить("Content-Type", "application/json; charset=utf-8");
73     HTTPЗапрос.Заголовки.Вставить("X-Trace-ID", ПараметрыЗапроса.TraceID);
74     HTTPЗапрос.УстановитьТелоИзСтроки(ТелоJSON, КодировкаТекста.UTF8);
75
76     Попытка
77         HTTPСоединение = Новый HTTPСоединение(
78             СерверБэкенда,
79             443,
80             "",
81             90, // Реальный замер - 42 сек на простом вопросе, запас на более сложные

```

```

82         Новый ЗащищенноеСоединениеOpenSSL()
83     );
84
85     Ответ = HTTPСоединение.ОтправитьДляОбработки(HTTPЗапрос);
86
87     Если Ответ.КодСостояния <> 200 Тогда
88         ЗаписьЖурналаРегистрации(
89             "NOPik.Сеть",
90             УровеньЖурналаРегистрации.Предупреждение,,,
91             СтрШаблон("Ошибка бэкенда nopikreport.com. Код: %1, Ответ: %2", Ответ.КодСостояния,
92             Ответ.ТелоКакСтроку());
93         Возврат;
94     КонецЕсли;
95
96     ЧтениеJSON = Новый ЧтениеJSON;
97     ЧтениеJSON.УстановитьСтроку(Ответ.ПолучитьТелоКакСтроку());
98     РезультатОтвета = ПрочитатьJSON(ЧтениеJSON, Истина);
99     ЧтениеJSON.Закрыть();
100
101     ТекстОтвета = РезультатОтвета.Получить("answer");
102
103     ЗаписьЖурналаРегистрации(
104         "NOPik.Сеть",
105         УровеньЖурналаРегистрации.Информация,,,
106         СтрШаблон("Ответ nopikreport.com получен (tier=%1, source=%2)",
107             РезультатОтвета.Получить("tier"), РезультатОтвета.Получить("source")),
108     );
109
110     ДоставитьОтветВОбсуждение(ПараметрыЗапроса, ТекстОтвета);
111
112     Искключение
113     ИнформацияОбОшибке = ИнформацияОбОшибке();
114     ЗаписьЖурналаРегистрации(
115         "NOPik.Сеть",
116         УровеньЖурналаРегистрации.Ошибка,,,
117         ПодробноеПредставлениеОшибки(ИнформацияОбОшибке)
118     );
119     КонецПопытки;
120
121 КонецПроцедуры
122
123 // Публикует готовый текст ответа обратно в чат пользователя.
124 //
125 // Работает только когда адаптер передал реальную ссылку на обсуждение
126 // (сегодня - адаптер «Бот»; сам объект «Бот» недоступен на этой платформе
127 // до 8.5.5, поэтому вызов СистемаВзаимодействия.СоздатьСообщение НЕ
128 // проверен живым выполнением - только компилятором). У адаптера «Команда
129 // формы» обсуждения нет: синхронное уведомление пользователя для него не
130 // входит в объем этой задачи, ответ остается только в журнале регистрации.

```

```

131 Процедура ДоставитьОтветВОбсуждение(ПараметрыЗапроса, ТекстОтвета)
132
133     ОбъектОбсуждения = ПараметрыЗапроса.ОбъектОбсуждения;
134
135     Если ОбъектОбсуждения = Неопределено Тогда
136         Возврат;
137     КонецЕсли;
138
139     ОтветСообщения = СистемаВзаимодействия.СоздатьСообщение(ОбъектОбсуждения);
140     ОтветСообщения.Текст = ТекстОтвета;
141     ОтветСообщения.Записать();

    КонецПроцедуры

#КонецОбласти

```

Живое тестирование: 6 runtime-ошибок, которые не поймал компилятор

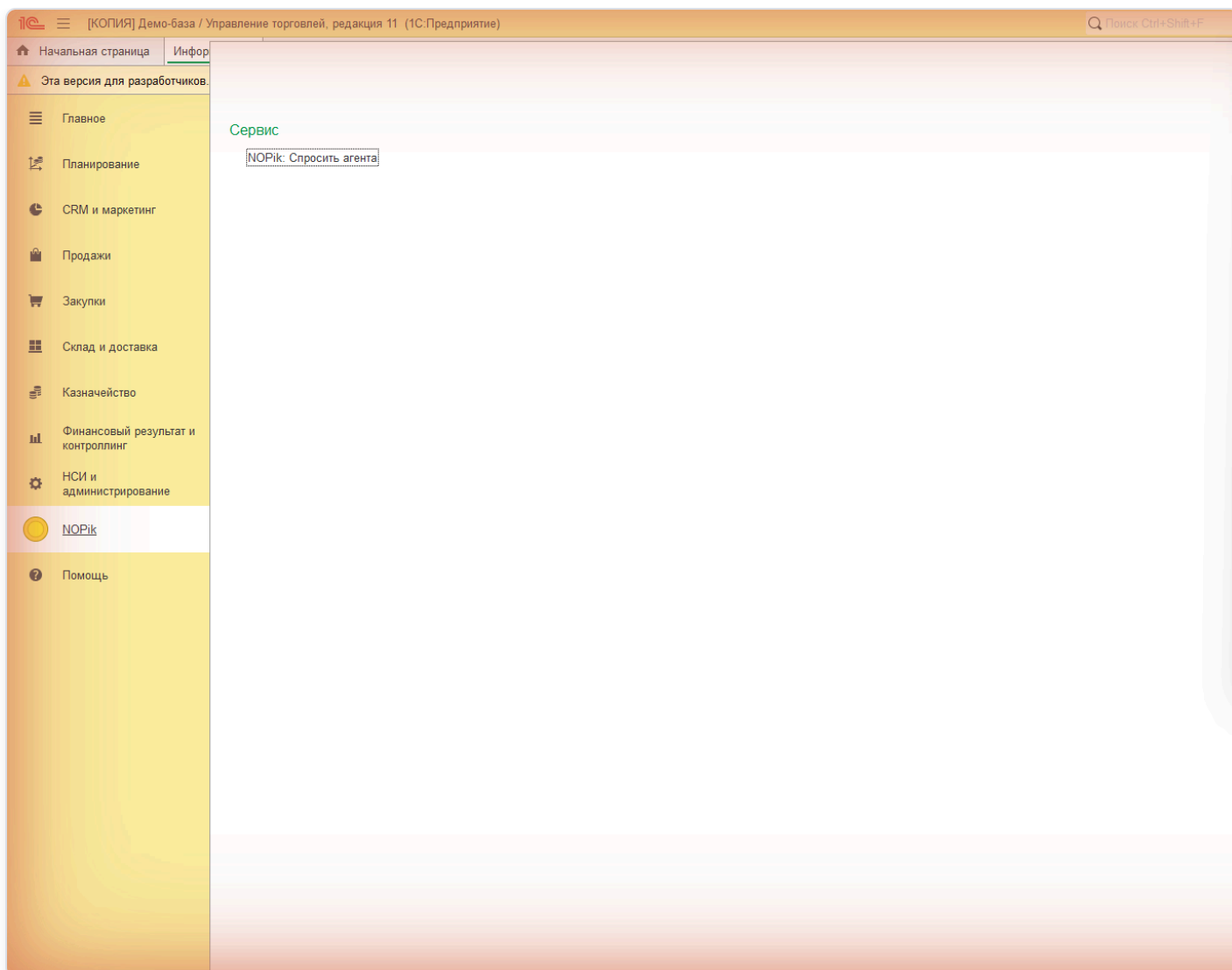
Код выше - уже финальная версия. Первая версия компилировалась в Конфигураторе без единой ошибки («Проверка модулей» - синтаксический контроль, не выполнение) и выглядела правильной на бумаге. Реальный прогон в 1С:Предприятии - не в Конфигураторе - вскрыл 6 отдельных ошибок, ни одну из которых статический контроль платформы не поймал: все они проявляются только в runtime, потому что связаны с типизацией, которая определяется во время исполнения, а не во время компиляции.

1. **Ссылка вместо Неопределено.** Новый ОписаниеОповещения ("Обработчик", Неопределено) падает с «Ошибка при вызове конструктора» - конструктору нужна настоящая ссылка на модуль, где определена процедура-обработчик.
2. **Клиент не может вызвать серверную процедуру двухконтекстного модуля по имени.** Платформа резолвит такой вызов к клиентскому экземпляру модуля, где серверных методов нет вообще - «Метод объекта не обнаружен» в runtime. Рабочий паттерн - чисто серверный модуль (ВызовСервера = Истина), вызываемый с клиента напрямую по имени.
3. **УникальныйИдентификатор - свойство, не метод.** Вызов со скобками, ...ТекущийПользователь().УникальныйИдентификатор(), падает с «Метод объекта не обнаружен» - скобки нужно убрать.
4. **Конструктору Массив нужно число, а не объект.** Новый Массив(КонтекстЗапроса) падает с «Ошибка при вызове конструктора» - единственный позиционный параметр

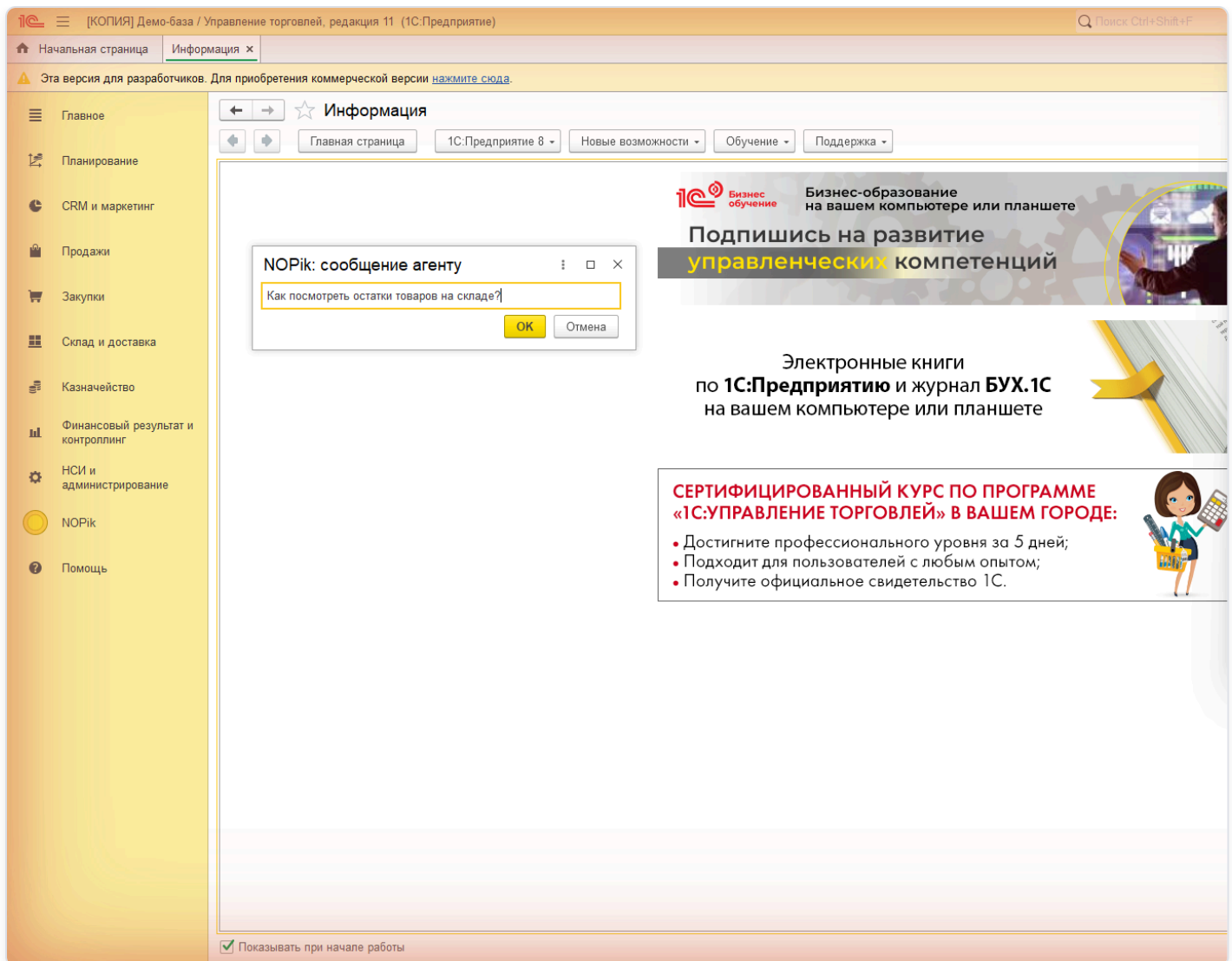
конструктора массив - это размерность, а не элемент. Массив параметров собирается через `.Добавить()`.

5. **Позиционные параметры** `HTTPСоединение` **сдвигают** таймаут. Пропуск одной лишней запятой при пропуске необязательных параметров (`Сервер`, `Порт`, `Пользователь`, `Пароль`, `ИнтернетПрокси`, *Таймаут*, `ЗащищенноеСоединение`, ...) сдвигает значение 90 в позицию `ЗащищенноеСоединение`, а объект `ЗащищенноеСоединениеOpenSSL` - в позицию `ИспользоватьАутентификациюОС`. Платформа ловит это только в runtime как «Конструктор не найден», потому что позиционные параметры типизируются во время исполнения.
6. **ПрочитатьJSON без второго параметра возвращает Структуру, а не Соответствие.** У Структуры метода `Получить(Ключ)` нет вообще - вызов `РезультатОтвета.Получить("answer")` падает с «Метод объекта не обнаружен». Второй параметр истина заставляет читать JSON-объекты как Соответствие, у которого этот метод есть.

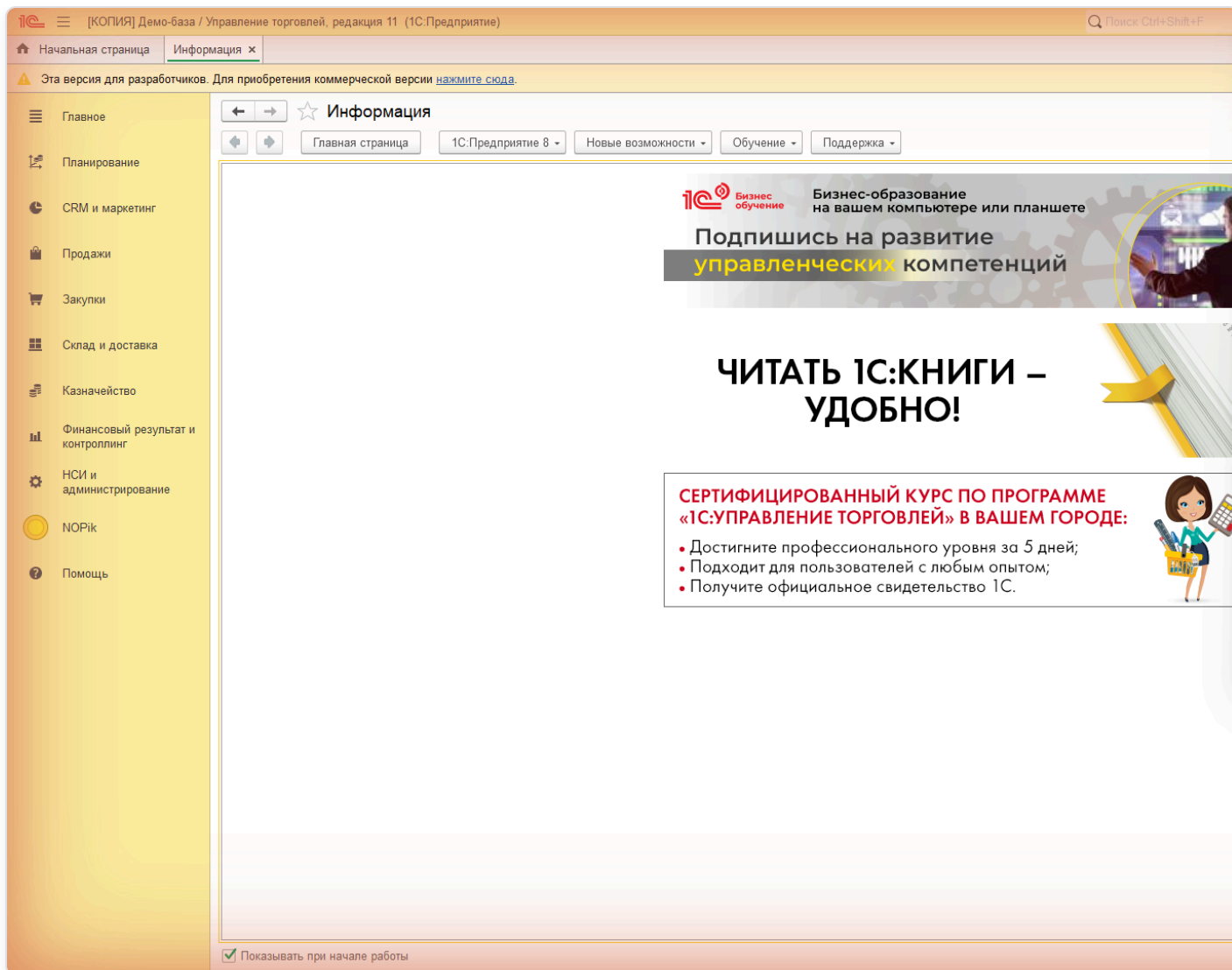
Каждая ошибка найдена и исправлена по одной и той же схеме: реальный клик «NOPIk: Спросить агента» в открытом сеансе 1С:Предприятия → «Метод объекта не обнаружен» / «Ошибка при вызове конструктора» в Журнале регистрации с номером строки модуля → правка → повторный клик. Ниже - скриншоты этого цикла на последней, успешной итерации.



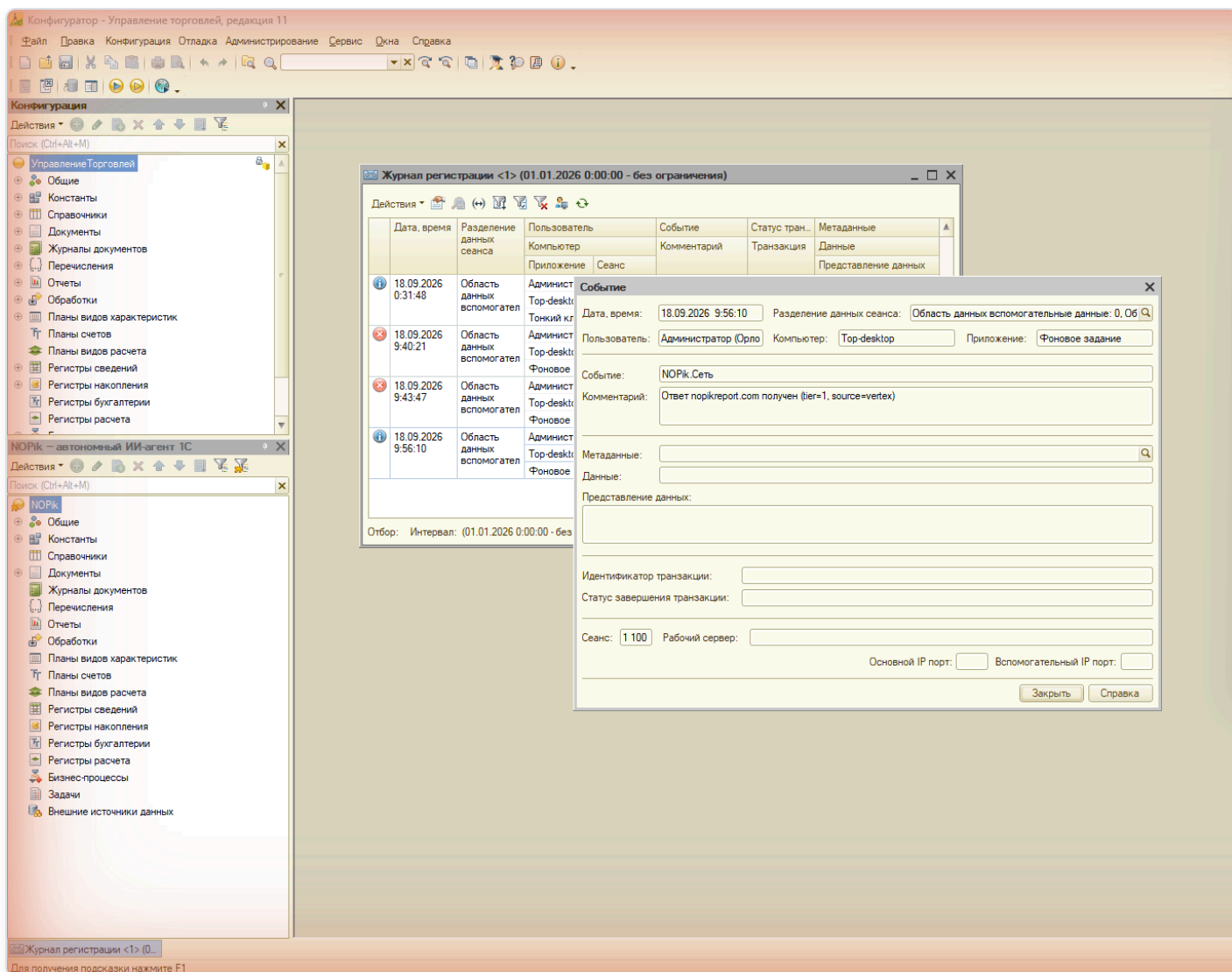
1С:Предприятие, живой сеанс: раздел «NOPIk» с командой «NOPIk: Спросить агента» - появился в командном интерфейсе только после добавления Подсистемы (см. раздел выше про роли и командный интерфейс).



Диалог адаптера «Команда формы»: реальный пользовательский ввод перед отправкой в фоновое задание.



Чистый возврат к странице «Информация» без диалога ошибки - все клиентские и раннесерверные баги из списка выше действительно устранены.



Журнал регистрации Конфигуратора, ~86 секунд после клика: «Ответ nopikreport.com получен (tier=1, source=vertex)» - фоновое задание реально дошло до `nopikreport.com/api/ask` и разобрало ответ. Это единственное надежное доказательство успеха для адаптера «Команда формы»: он не выводит ответ обратно в интерфейс, только протоколирует (см. ограничение выше про адаптер «Бот»).

Честное ограничение: адаптер «Команда формы» не показывает текст ответа пользователю - он существует ровно для того, чтобы можно было проверить всю цепочку до появления объекта метаданных «Бот» в 8.5.5. Единственное наблюдаемое сегодня доказательство успеха - запись в Журнале регистрации выше. Когда «Бот» станет доступен, тот же контракт `НОПик_Сеть.ПоставитьЗадачуНаОтправку` доставит текст ответа прямо в чат Системы взаимодействия - код для этого уже написан и ждет платформы (см. `Bots/НОПик/Ext/Module.bs1`).

Учебная ветка: **свой оркестратор на Python и LangGraph**

Важное уточнение, прежде чем читать этот раздел: то, что сейчас реально отвечает на сообщение бота - живой сервис `nopikreport.com`, показанный в разделе выше, а не граф ниже. Этот граф - не описание того, что происходит на стороне `nopikreport.com` (туда мы не заглядываем и не должны), а самостоятельный учебный путь: как построить **свой** оркестратор, если вам нужна логика поверх **ваших собственных** данных 1С (а не общий

вопрос, на который отвечает готовый сервис). Ниже - минимальная, но полностью функциональная архитектурная реализация графа FSM такого агента на базе Python и контрактов Pydantic; она не подключена к боевому пути и развивается отдельно в следующих статьях цикла.

Ключевые узлы графа:

- `analyze_intent_node` - определяет намерение пользователя (построение отчета, диагностика ошибки, общий вопрос) и формирует план шагов;
- `plan_executor_node` - выполняет сформированный план и готовит текст ответа;
- `deliver_to_1c_node` - отправляет ответ обратно в Систему взаимодействия 1С через обратный webhook.

Python — граф состояний оркестратора (LangGraph)

UTF-8

[Открыть файл](#)

```
1 from typing import TypedDict, Annotated, Literal
2 from pydantic import BaseModel, Field
3 from langgraph.graph import StateGraph, START, END
4 from langgraph.checkpoint.memory import MemorySaver
5
6 # 1. Схема входного контракта сообщения из 1С
7 class Incoming1CMessage(BaseModel):
8     message_id: str = Field(..., alias="MessageID")
9     conversation_id: str = Field(..., alias="ConversationID")
10    author_id: str = Field(..., alias="AuthorID")
11    text: str = Field(..., alias="Text")
12    trace_id: str = Field(..., alias="TraceID")
13
14 # 2. Состояние агента (граф состояний FSM)
15 class AgentState(TypedDict):
16     conversation_id: str
17     trace_id: str
18     user_query: str
19     intent: str
20     plan: list[str]
21     current_step: int
22     final_response: str
23     error: str | None
24
25 # 3. Узлы графа состояний (Nodes)
26 def analyze_intent_node(state: AgentState) -> dict:
27     """Определение намерения пользователя: отчет, анализ ошибки, консультация."""
28     query = state["user_query"].lower()
29
30     if "остат" in query or "склад" in query or "отчет" in query:
31         return {"intent": "REPORT_GENERATION", "plan": ["DISCOVER_METADATA", "BUILD_
32     elif "ошибка" in query or "лог" in query or "журнал" in query:
```

```

33         return {"intent": "DIAGNOSTICS", "plan": ["READ_EVENT_LOG", "ANALYZE_TRACE"]}
34     else:
35         return {"intent": "GENERAL_QUERY", "plan": ["GENERATE_ANSWER"]}
36
37 def plan_executor_node(state: AgentState) -> dict:
38     """Выполнение сформированного плана действий."""
39     intent = state["intent"]
40     if intent == "REPORT_GENERATION":
41         response = (
42             f"Запрос на построение отчета принят (TraceID: {state['trace_id']}).\n"
43             f"План выполнения: {' '.join(state['plan'])}.\n"
44             f"Исследую остатки товаров через MCP-контур..."
45         )
46     else:
47         response = f"Обработка запроса: '{state['user_query']}'."
48
49     return {"final_response": response}
50
51 def deliver_to_1c_node(state: AgentState) -> dict:
52     """Отправка сформированного ответа обратно в Систему взаимодействия 1С."""
53     # Вызов обратного Webhook/API 1С для отправки сообщения в чат conversation_id
54     print(f"[1C Webhook] Отправка в диалог {state['conversation_id']}: {state['final_response']}")
55     return {}
56
57 # 4. Сборка графа
58 workflow = StateGraph(AgentState)
59
60 workflow.add_node("analyze_intent", analyze_intent_node)
61 workflow.add_node("plan_executor", plan_executor_node)
62 workflow.add_node("deliver_to_1c", deliver_to_1c_node)
63
64 workflow.add_edge(START, "analyze_intent")
65 workflow.add_edge("analyze_intent", "plan_executor")
66 workflow.add_edge("plan_executor", "deliver_to_1c")
67 workflow.add_edge("deliver_to_1c", END)
68
69 # Чекпоинтер сессий по conversation_id для отказоустойчивости
70 memory_checkpointer = MemorySaver()
71 agent_app = workflow.compile(checkpointer=memory_checkpointer)

```

Узлы связываются в StateGraph с чекпоинтером MemorySaver (в production заменяется на PostgresSaver - тема статьи 6 цикла), что дает агенту устойчивость к сбоям сети и падениям процесса ровно в духе HSM-чекпоинтинга, описанного выше.

🍌 **Итоги: дорожная карта цикла**

Дальше цикл переходит к остальным компонентам корпоративной инженерии ИИ-агентов. Следующие материалы серии:

- **Статья 2. Паттерны рассуждений и мультиагентная оркестрация enterprise-процессов 1С.** Декомпозиция задач NOPIk на роли: Супервайзер, Аналитик метаданных, Сборщик СКД. Паттерны ReAct и Tree-of-Thoughts.
- **Статья 3. Универсальные контракты и протокол MCP.** Развертывание MCP-сервера 1С, валидация через Instructor/Pydantic, кликабельные навигационные ссылки `e1cib/data/....`
- **Статья 4. Надежность и качество.** TDD для агента: модульные тесты YAxUnit, сценарии Vanessa Automation, статический анализ BSL Language Server.
- **Статья 5. Изоляция и DevSecOps.** Контейнеризация в Docker/Kubernetes, безопасные песочницы, защита от Path Traversal.
- **Статья 6. Корпоративная память агентов.** Двухуровневая память: ConversationID против семантического поиска в Qdrant, шины RabbitMQ и 1С:Шина.
- **Статья 7. Безопасность, мониторинг и наблюдаемость.** Threat Modeling по OWASP ASI Top 10, Human-in-the-Loop прямо в чате, трейсы в Grafana/ClickHouse.

Исходные коды расширения NOPIk и примеры конфигурации графов доступны в репозитории проекта.

Какие узкие места вы видите в переходе от «монолитных» обработок с ИИ к архитектуре на графах состояний в вашей инфраструктуре 1С? Делитесь опытом и вопросами в комментариях - разберем в следующих статьях цикла.

Теги: 1С:Предприятие 8.3, Система взаимодействия, Объект Бот, ИИ-агент, LLM, LangGraph, FSM, HSM, Граф состояний, MCP, ibcmd, BSL, Фоновые задания, Zero-Footprint, Автоматизация 1С, Python, Pydantic, Enterprise AI, NOPIk